

LangChain Quick Reference Guide

⚡ Essential Imports

```
# Core components
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage,
AIMessage
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser,
PydanticOutputParser
from langchain_core.tools import tool
from langchain.agents import create_react_agent, AgentExecutor

# For RAG
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings

# For memory
from langchain.memory import ConversationBufferMemory

# Environment
from dotenv import load_dotenv
import os
```

1 Setup (Always First!)

```
load_dotenv() # Load from .env file

# Check API key
if not os.environ.get("OPENAI_API_KEY"):
    raise ValueError("OPENAI_API_KEY not set")

# Initialize LLM
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)
```

2 Messages

```
# Three types of messages
system_msg = SystemMessage(content="You are a Python expert")
human_msg = HumanMessage(content="What is a decorator?")
ai_msg = AIMessage(content="A decorator is a function that...")

# Use as list for conversations
messages = [system_msg, human_msg]
response = llm.invoke(messages)
messages.append(AIMessage(content=response.content))
```

3 Simple LLM Call

```
# Single invocation
response = llm.invoke([HumanMessage(content="Hello")])
print(response.content)

# Streaming
for chunk in llm.stream([HumanMessage(content="Hello")]):
    print(chunk.content, end="", flush=True)

# Batch processing
responses = llm.batch([
    HumanMessage(content="Q1"),
    HumanMessage(content="Q2"),
])
```

4 Prompts

```
# Simple template
prompt = PromptTemplate(
    template="Tell me a joke about {topic}",
    input_variables=["topic"]
)
formatted = prompt.invoke({"topic": "Python"})

# Chat template (with roles)
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant"),
    ("user", "Explain {concept}")
])
formatted = prompt.invoke({"concept": "decorators"})

# Partial (pre-fill variables)
partial = prompt.partial(system_prompt="Be concise")
```

5 Output Parsing

```
# String parser
parser = StrOutputParser()
chain = prompt | llm | parser
result = chain.invoke({"topic": "Python"}) # Returns string

# Pydantic parser (RECOMMENDED)
from pydantic import BaseModel, Field

class Joke(BaseModel):
    setup: str = Field(description="Setup of joke")
    punchline: str = Field(description="Punchline of joke")

parser = PydanticOutputParser(pydantic_object=Joke)
prompt = PromptTemplate(
    template="Generate a joke.\n{format_instructions}",
    input_variables=[],
    partial_variables={
        "format_instructions": parser.get_format_instructions()
    }
)
chain = prompt | llm | parser
result = chain.invoke({}) # Returns Joke object
```

6 Chains (LCEL)

```

# Basic chain
chain = prompt | llm | parser

# With custom function
def add_prefix(text):
    return f"💡 {text}"

chain = prompt | llm | parser | add_prefix

# Parallel execution
from langchain_core.runnables import RunnableParallel

parallel = RunnableParallel(
    title=title_chain,
    description=desc_chain,
    keywords=keywords_chain
)
result = parallel.invoke({"topic": "AI"})
# Returns: {"title": "...", "description": "...", "keywords": "..."}

# Invoking chains
result = chain.invoke({"topic": "Python"})
for chunk in chain.stream({"topic": "Python"}):
    print(chunk, end="")
results = chain.batch([{"topic": "Python"}, {"topic": "Java"}])

```

7 Pydantic Models

```
from pydantic import BaseModel, Field, field_validator

class Article(BaseModel):
    title: str = Field(description="Article title")
    author: str = Field(description="Author name")
    tags: list[str] = Field(default_factory=list)
    rating: float = Field(ge=1, le=10)

    @field_validator('rating')
    @classmethod
    def rating_valid(cls, v):
        if v < 1 or v > 10:
            raise ValueError('Rating must be 1-10')
        return v

# Create instance
article = Article(
    title="Learning Python",
    author="John Doe",
    tags=["python", "learning"],
    rating=9.5
)

# Convert to dict/JSON
article_dict = article.model_dump()
article_json = article.model_dump_json()
```

8 Tools

```
# Create tool with @decorator
@tool
def calculator(expression: str) -> str:
    """Evaluate math expressions"""
    return str(eval(expression))

@tool
def get_weather(city: str) -> str:
    """Get weather for a city"""
    return f"Sunny in {city}"

# Bind tools to LLM
tools = [calculator, get_weather]
llm_with_tools = llm.bind_tools(tools)

# Check if LLM called tools
response = llm_with_tools.invoke([HumanMessage(content="What's 5*3?")])
if response.tool_calls:
    for call in response.tool_calls:
        name = call['name']
        args = call['args']
        # Execute tool manually
        if name == "calculator":
            result = calculator(args['expression'])
```

Agents

```

from langchain.agents import create_tool_calling_agent, AgentExecutor

# Create agent
agent = create_tool_calling_agent(
    llm,
    tools,
    ChatPromptTemplate.from_messages([
        ("system", "You are a helpful assistant"),
        ("human", "{input}")
    ])
)

# Create executor
executor = AgentExecutor.from_agent_and_tools(
    agent=agent,
    tools=tools,
    max_iterations=10,
    verbose=True
)

# Run agent
result = executor.invoke({"input": "What is (5 + 3) * 2?"})
print(result['output'])

# Stream agent execution
for step in executor.stream({"input": "Calculate 5*3"}):
    if "actions" in step:
        print(f"Calling: {step['actions']}")
    elif "final_output" in step:
        print(f"Answer: {step['final_output']}")

```

10 RAG (Retrieval-Augmented Generation)


```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_core.documents import Document

# 1. Create documents
documents = [
    Document(page_content="Python is a language..."),
    Document(page_content="AI is artificial intelligence..."),
]

# 2. Split into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=200,
    chunk_overlap=50
)
chunks = splitter.split_documents(documents)

# 3. Create vector store
embeddings = OpenAIEmbeddings()
vector_store = FAISS.from_documents(chunks, embeddings)

# 4. Create retriever
retriever = vector_store.as_retriever(search_kwargs={"k": 3})

# 5. Use in chain
def format_docs(docs):
    return "\n\n".join([d.page_content for d in docs])

rag_prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer based on context"),
    ("human", "Context: {context}\n\nQuestion: {question}")
])

rag_chain = (
    {"context": retriever | format_docs, "question": lambda x: x}
    | rag_prompt
    | llm
)
```

```
    | StrOutputParser()
)

answer = rag_chain.invoke("What is Python?")
```

11 Memory

```
from langchain.memory import ConversationBufferMemory

# Create memory
memory = ConversationBufferMemory(return_messages=True)

# Add to memory
memory.chat_memory.add_user_message("What is Python?")
memory.chat_memory.add_ai_message("Python is a language...")

# Retrieve memory
history = memory.load_memory_variables({})
messages = history["history"] # List of messages

# Use in prompt
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are helpful"),
    ("placeholder", "{chat_history}"),
    ("human", "{input}")
])

# Pass memory to LLM
formatted = prompt.invoke({
    "chat_history": messages,
    "input": "What can I build with it?"
})

response = llm.invoke(formatted)
```

1 2 Temperature Guide

```
# Temperature = 0 (Deterministic)
llm = ChatOpenAI(temperature=0)
# Use for: Facts, Q&A, code generation, customer service
# Same input → Same output

# Temperature = 0.7 (Balanced)
llm = ChatOpenAI(temperature=0.7)
# Use for: General conversations, creative but grounded
# Good default for most cases

# Temperature = 1.5 (Creative)
llm = ChatOpenAI(temperature=1.5)
# Use for: Brainstorming, creative writing, idea generation
# High variation between calls
```

1 3 Error Handling

```
from langchain_core.output_parsers import OutputParserException

try:
    # Code that might fail
    response = llm.invoke(messages)
    parsed = parser.invoke(response)
except OutputParserException as e:
    print(f"Parsing error: {e}")
    # Handle gracefully
except Exception as e:
    print(f"Unexpected error: {e}")
    # Retry or use fallback
```

1 4 Complete Minimal Example

```

from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv
import os

# Setup
load_dotenv()
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

# Create chain
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant"),
    ("user", "Tell me a joke about {topic}")
])

parser = StrOutputParser()

chain = prompt | llm | parser

# Use
result = chain.invoke({"topic": "Python"})
print(result)

```



When to Use What

Goal	Tool
Simple LLM call	<code>llm.invoke()</code>
Multi-step pipeline	LCEL chains with <code> </code>
Dynamic prompts	<code>PromptTemplate</code>
Structured output	<code>PydanticOutputParser</code>

External functions	@tool decorator
Multi-step reasoning	Agents with AgentExecutor
Current knowledge	RAG with vector store
Conversation context	ConversationBufferMemory

Best Practices

1. **Always use environment variables** for API keys (never hardcode)
2. **Set temperatures appropriately**: 0 for facts, 0.7 for general, 1.5+ for creative
3. **Use Pydantic for output validation** - catches LLM errors early
4. **Implement timeouts** on all API calls to prevent hanging
5. **Log everything** - queries, responses, errors for debugging
6. **Test with small models first** - cheaper for development
7. **Use streaming** for better UX - show responses in real-time
8. **Implement retries** - LLM APIs can be flaky
9. **Cache embeddings** - expensive to recompute for RAG
10. **Monitor token usage** - it adds up fast in production

LCEL Operators Cheat Sheet

```
# Pipe operator (sequence)
chain = step1 | step2 | step3

# Parallel execution
from langchain_core.runnables import RunnableParallel
parallel = RunnableParallel(a=chain1, b=chain2)

# Conditional routing
from langchain_core.runnables import RunnableIfElse
conditional = RunnableIfElse(condition, if_true, if_false)

# Lambda functions
from langchain_core.runnables import RunnableLambda
transform = RunnableLambda(lambda x: x.upper())

# Pass-through (keep original)
from langchain_core.runnables import RunnablePassthrough
keep = RunnablePassthrough()
```



Common Patterns

Pattern 1: Prompt → LLM → Parser

```
chain = prompt_template | llm | output_parser
```

Pattern 2: RAG

```
chain = {"context": retriever | format, "question": pass_through} |
prompt | llm | parser
```

Pattern 3: Agent Loop

```
agent = create_agent(llm, tools, prompt)
executor = AgentExecutor(agent, tools)
executor.invoke({"input": query})
```

Pattern 4: Memory-enabled Conversation

```
prompt = ChatPromptTemplate([system, chat_history_placeholder, human])
formatted = prompt.invoke({"chat_history": memory.load(), "input":
user_input})
response = llm.invoke(formatted)
memory.add_messages([HumanMessage(user_input), AIMessage(response)])
```

Model Recommendations

Task	Recommended Model	Temperature
Customer Support	gpt-3.5-turbo	0.3
Content Generation	gpt-4-turbo	0.8
Code Generation	gpt-4-turbo	0.2
Brainstorming	gpt-4-turbo	1.2
Information Extraction	gpt-3.5-turbo	0
General Purpose	gpt-3.5-turbo	0.7
Cost-Sensitive	gpt-3.5-turbo	default



Pro Tips

1. Use f-strings in prompts for debugging

```
template = f"Explain {concept} to a {audience}"
```

2. Test prompts before using in production

```
test_result = chain.invoke(test_input)
assert "expected" in test_result.lower()
```

3. Use model_dump_json() for API responses

```
json_str = model.model_dump_json(indent=2)
```

4. Chain errors are usually parsing errors

```
# Add validation to catch early
parser = PydanticOutputParser(pydantic_object=MyModel)
```

5. Stream for better perceived performance

```
# User sees response immediately, not 3-5 second wait
for chunk in chain.stream(input):
    print(chunk, end="", flush=True)
```

Print this page (Ctrl+P) or save as PDF for offline reference!